Cadenas, Archivos y Funciones Avanzadas en Python

3/3 – De cadenas y `string.h` a `open()` y `stdio.h`

Informática y Programación

UNCuyo – Facultad de Ingeniería

9 de septiembre de 2026

Índice de Contenidos

1. 1. Cadenas: Indexado y Slicing
2. 2. Cadenas Inmutables
3. 3. Métodos de Cadena
4. 4. Manejo de Archivos
5. 5. Funciones Avanzadas
6. 6. Caso Aplicado y Resumen

El cierre de la serie

Dónde estamos

1. Fundamentos: sintaxis, tipos, control, E/S, funciones.
2. Estructuras de datos: listas, tuplas y diccionarios.
3. **Cadenas, archivos y funciones avanzadas** (ésta).

Para qué sirve esta última parte

Cierra lo que falta para el ejercicio 7 (Wordle, manipulación de cadenas) y resuelve directamente el ejercicio 9 del TP4: leer un archivo de texto y calcular estadísticas sobre su contenido.

Indice de Contenidos

1. 1. Cadenas: Indexado y Slicing
2. 2. Cadenas Inmutables
3. 3. Métodos de Cadena
4. 4. Manejo de Archivos
5. 5. Funciones Avanzadas
6. 6. Caso Aplicado y Resumen

Acceder a un carácter

Misma idea, mismo símbolo

En los dos lenguajes una cadena se indexa como un arreglo: posición 0 es el primer carácter.

C

```c
char s[] = "Hola";
printf("%c\n", s[0]);
// Imprime: H
```

Python

```python
s = "Hola"
print(s[0])
# Imprime: H
```

Obtener una subcadena: `slicing`

Acá empiezan a distanciarse

Python tiene `slicing` incorporado al lenguaje: `s[inicio:fin]`. En C no existe — hay que armar la subcadena a mano, carácter por carácter.

C

```c
char s[] = "Programacion";
char sub[4];
for (int i = 0; i < 3; i++)
    sub[i] = s[2 + i];
sub[3] = '\0';
// sub = "ogr"
```

Python

```python
s = "Programacion"
sub = s[2:5]
# sub = "ogr"
```

Bonus: invertir una cadena

`s[::-1]`: paso negativo

El tercer valor del slice es el **paso**. Con paso `-1`, Python recorre la cadena de atrás para adelante — útil para, por ejemplo, verificar palíndromos (ya lo hicieron en C, en el TP3, con recursión).

C

```c
int n = strlen(s);
for (int i = n - 1; i >= 0; i--)
    putchar(s[i]);
// recorrido manual
```

Python

```python
s = "Hola"
print(s[::-1])
# Imprime: aloH
```

Indice de Contenidos

1. 1. Cadenas: Indexado y Slicing
2. 2. Cadenas Inmutables
3. 3. Métodos de Cadena
4. 4. Manejo de Archivos
5. 5. Funciones Avanzadas
6. 6. Caso Aplicado y Resumen

Las cadenas en Python no se pueden modificar

Contraintuitivo viniendo de C

En C, `s` es un arreglo de `char`: cada posición se puede reescribir libremente (`s[0] = 'X';`). En Python, un `str` es **inmutable** — una vez creado, ningún carácter se puede cambiar en el lugar. Cualquier operación que "modifica" una cadena en realidad **crea una cadena nueva**.

Por qué importa

Es la razón por la que `s = s + "algo"` o `s[2:5]` no tocan la cadena original: siempre devuelven una cadena distinta.

El error típico y cómo evitarlo

C (válido)

```c
char s[] = "Hola";
s[0] = 'h';
printf("%s\n", s);
// Imprime: hola
```

Python (error)

```python
s = "Hola"
s[0] = 'h'
# TypeError: 'str' object
# does not support item
# assignment
```

```python
s = "Hola"
s = "h" + s[1:]    # se arma una cadena nueva
print(s)           # Imprime: hola
```

Indice de Contenidos

1. 1. Cadenas: Indexado y Slicing
2. 2. Cadenas Inmutables
3. 3. Métodos de Cadena
4. 4. Manejo de Archivos
5. 5. Funciones Avanzadas
6. 6. Caso Aplicado y Resumen

Lo que en C es una función suelta, en Python es un método

`cadena.metodo()` en vez de `funcion(cadena)`

`<string.h>` da funciones que reciben la cadena como parámetro; Python las expone como métodos del propio `str`.

Python	**Equivalente aproximado en C**
`s.upper()` / `s.lower()`	bucle manual con `toupper`/`tolower`
`s.strip()`	recorrer y descartar espacios en los bordes
`s.split(sep)`	`strtok()` (con estado global, más delicado)
`sep.join(lista)`	concatenar a mano con `strcat()`
`s.count(sub)`	bucle manual buscando ocurrencias
`s.replace(a, b)`	sin equivalente directo, se arma a mano

`split()` y `join()`: separar y armar

Los inversos el uno del otro

`split()` separa una cadena en una lista; `join()` hace lo opuesto: une una lista en una única cadena, con un separador.

```
frase = "el perro come carne"
palabras = frase.split()              # ['el', 'perro', 'come', 'carne'

reconstruida = "-".join(palabras) # "el-perro-come-carne"
```

En C, la función equivalente es `strtok()` — funciona, pero modifica la cadena original y guarda estado entre llamadas, lo que la hace más propensa a errores que `split()`.

Indice de Contenidos

1. 1. Cadenas: Indexado y Slicing
2. 2. Cadenas Inmutables
3. 3. Métodos de Cadena
4. 4. Manejo de Archivos
5. 5. Funciones Avanzadas
6. 6. Caso Aplicado y Resumen

Abrir y cerrar: `open()` + `with` vs. `fopen/fclose`

El mismo riesgo que con `malloc/free`

En C, todo `fopen()` exitoso necesita su `fclose()` — olvidarlo dejar el archivo abierto es un descuido parecido a olvidar un `free()` en el TP3 (Parte 2).

El bloque `with` de Python

`with open(...) as f:` cierra el archivo automáticamente al salir del bloque, **incluso si ocurre un error adentro**. No hay equivalente automático en C: el `fclose()` hay que escribirlo siempre a mano.

Abrir, leer todo el contenido y cerrar

C

```c
FILE *f = fopen("a.txt", "r");
if (f == NULL) {
    return 1; // error
}
char buf[256];
fread(buf, 1, 255, f);
fclose(f); // obligatorio
```

Python

```python
with open("a.txt") as f:
    contenido = f.read()

# el archivo ya esta
# cerrado aca afuera
```

Leer línea por línea

Iterar el archivo directamente

Un archivo abierto con `open()` es iterable: cada vuelta del `for` entrega la siguiente línea, sin manejar buffers a mano.

C

```c
char linea[256];
FILE *f = fopen("a.txt", "r");
while (fgets(linea, 256, f)) {
    printf("%s", linea);
}
fclose(f);
```

Python

```python
with open("a.txt") as f:
    for linea in f:
        print(linea, end="")
```

Indice de Contenidos

1. 1. Cadenas: Indexado y Slicing
2. 2. Cadenas Inmutables
3. 3. Métodos de Cadena
4. 4. Manejo de Archivos
5. 5. Funciones Avanzadas
6. 6. Caso Aplicado y Resumen

Parámetros con valor por defecto

Algo que C no tiene

Python permite que un parámetro tenga un valor si quien llama no lo especifica. En C no existe esto: todos los argumentos se pasan siempre, o se define una función distinta para cada caso.

```
def calcular_tarifa(horas, tarifa_por_hora=1000):
    return horas * tarifa_por_hora

calcular_tarifa(3)          # usa tarifa_por_hora=1000 -> 3000
calcular_tarifa(3, 2000)    # la pisa -> 6000
```

Repaso rápido: el retorno múltiple con tuplas (`return a, b`) ya lo vimos en la presentación 2 — es la otra herramienta clave para funciones más expresivas que en C.

Indice de Contenidos

1. 1. Cadenas: Indexado y Slicing
2. 2. Cadenas Inmutables
3. 3. Métodos de Cadena
4. 4. Manejo de Archivos
5. 5. Funciones Avanzadas
6. 6. Caso Aplicado y Resumen

Caso aplicado: palabra más repetida (TP4, ej. 9)

Contar frecuencia de palabras con un diccionario

Recorrer las palabras y, por cada una, sumar 1 a su contador en un diccionario — `dict.get(clave, 0)` devuelve 0 si la clave todavía no existe, evitando un `if` extra.

```
frecuencia = {}
for palabra in palabras:
    if len(palabra) > 6:
        frecuencia[palabra] = frecuencia.get(palabra, 0) + 1

# la(s) palabra(s) con mayor frecuencia:
maximo = max(frecuencia.values())
mas_repetidas = [p for p, c in frecuencia.items() if c == maximo]
```

Tabla Comparativa Final

Característica	**Lenguaje C**	**Lenguaje Python**
Cadenas	Arreglo de `char` mutable, con `'\0'`	`str` inmutable, con métodos propios
Archivos	`fopen`/`fclose` manual	`with open(...)` con cierre automático
Manejo de recursos	Responsabilidad del programador	Garbage collector + `with`

Con esto ya tenés todo el TP4

Parte A

Análisis, entradas/salidas y diseño de funciones — ya lo resolvieron en C y pseudocódigo.

Parte B

Variables, control de flujo, funciones (presentación 1); listas, tuplas y diccionarios (presentación 2); cadenas, archivos y funciones avanzadas (ésta) — todo lo necesario para los ejercicios 5 a 9.

¡A resolver el TP4!